Supplementary material for: Closed-form chance constraints for fast multi-UAV replanning under uncertain execution

Cao-Phuc Ha, Anh Nguyen Vu Minh, Phu-Nguyen Le

Abstract

This document contains the parts of the evaluation that the main paper only summarises: the stopping rule and its two delay costs with their properties, the scenario generator and the fitted error models, the exact-execution benchmark of the stopping rules, the full results of study 1, and further results of study 2 per regime and in tier C. Section, equation and table numbers without the prefix S refer to the main paper.

S1. Stopping rule and delay costs

S1.1. Learned stopping rule

Every 5 ms, the stopping rule compares the predicted gain of searching for another $H = 10$ ms with the cost of doing so. With $\widehat{\Delta J}$ as the predicted gain, the search stops at time τ if

$$\widehat{\Delta J} \;\le\; C_{\text{delay}}(\tau) + \mu W H. \tag{S1}$$

The gain is predicted from the search history by a Poisson generalised linear model with a log link [1] and a small L_2 penalty. Its eleven features describe the elapsed time, the number of iterations, the recent improvements over 5 ms, over 20 ms and over the second half of the elapsed time, the time since the last improvement, the total improvement over the repaired plan, the unassigned share of the reward, the numbers of tasks and UAVs, and the iteration rate. The target is the improvement in the next 10 ms in percent of W. The model is fitted on 500 ms search trajectories of development scenarios only, and a prediction costs one dot product. The search loop enforces the hard time budget B independently of the stopping rule. The price of computation is $\mu = 10^{-5}$ per millisecond.

S1.2. Deterministic delay cost

The delay cost is computed from the best plan found so far. A late start of a route is partly absorbed by the waiting times along it, in the same way as the forward time slack of routing with time windows [2]. The tolerance tol_j of a task is the margin between its finish time and its deadline, plus all waiting times on the route up to this task. A task is lost when the start shift $\sigma_j(\tau)$ of its route exceeds its tolerance. The start shift is zero for a UAV that is still busy, and it equals the waiting time until the publication for an idle UAV. With $v_j = \alpha w_j - \delta\,\mathbb{1}[j \text{ changes UAV}]$ as the value of task j in the objective and $\Lambda(\tau) = \sum_j v_j\,\mathbb{1}[\sigma_j(\tau) > \text{tol}_j]$, the delay cost of searching H milliseconds longer is $C_{\text{delay}}(\tau) = \Lambda(\tau + H) - \Lambda(\tau)$.

Proposition S1. *For a feasible plan executed exactly, (i) $\Lambda(\tau)$ is the value lost through missed deadlines if every UAV follows its route, (ii) the loss of the objective is at most $\Lambda(\tau)$ if late tasks are skipped, and (iii) $\Lambda(\tau)$ does not decrease with τ and is zero while all UAVs are busy with their committed tasks. Hence C_{delay} is non-negative and vanishes while no UAV is waiting.*

Proof. A route that starts later by some shift reaches its first task later by the same amount. Each waiting time absorbs part of the delay, so the delay at the end of a task equals the start shift minus all earlier waiting times, if this difference is positive. The task therefore misses its deadline exactly when the start shift exceeds its tolerance. Waiting consumes no energy, which proves (i). If late tasks are skipped, the triangle inequality ensures that every later task is reached no later and with no more energy. All tasks that are on time in (i) remain on time, and the flown distance, hence the energy term γE, does not grow. The objective therefore loses the values v_j of the skipped tasks and nothing else, which is at most $\Lambda(\tau)$ and proves (ii). Part (iii) holds because the start shift does not decrease with τ and every tolerance is non-negative in a feasible plan.

S1.3. Execution-aware delay cost

Under uncertain execution, task j is lost when $\sigma_j(\tau) + e_j > \mathrm{tol}_j$, where e_j is its finish-time error. The execution-aware cost replaces the indicator in Λ by the probability of this event. The error of task j is modelled as $e_j \sim \mathcal{N}(m(h_j), s(h_j)^2)$, where $h_j = f_j - t_0$ is its lead time, the nominal finish time minus the time of the planning snapshot. The mean and the spread are linear in the lead time,

$$m(h) = b_0 + b_1 h, \qquad s(h) = s_0 + s_1 h, \tag{S2}$$

because errors accumulate along a route. The expected dissemination latency $\bar{L}$ is added to the start shift. The expected loss and the execution-aware delay cost are

$$\Lambda_{\mathrm{R}}(\tau) = \sum_j v_j \, \Phi\!\left(\frac{\sigma_j(\tau) + \bar{L} + m(h_j) - \mathrm{tol}_j}{s(h_j)} \right), \qquad C_{\mathrm{R}}(\tau) = \Lambda_{\mathrm{R}}(\tau + H) - \Lambda_{\mathrm{R}}(\tau), \tag{S3}$$

where Φ is the standard normal distribution function. Search and predictor are unchanged, and C_{R} replaces C_{delay} in Eq. (S1).

Proposition S2. *(i) With $b_0 = b_1 = s_0 = s_1 = 0$ and $\bar{L} = 0$, $\Lambda_{\mathrm{R}} = \Lambda$, and the stopping rule takes exactly the decisions of the deterministic one. (ii) $\Lambda_{\mathrm{R}}(\tau)$ does not decrease with τ, so $C_{\mathrm{R}} \geq 0$. (iii) Under the error model, $\Lambda_{\mathrm{R}}(\tau)$ is the expected value of the tasks whose deadline is missed if every UAV follows its route.*

Proof. With zero spread, Φ is read as the indicator of a positive argument, which gives (i). The start shift does not decrease with τ and Φ is non-decreasing, which gives (ii). Part (iii) follows from the proof of Proposition S1, because task j misses its deadline exactly when its delay at the finish, $\sigma_j + e_j$ minus the absorbed waiting time, is positive, and the expectation of the indicator of this event is the probability in Eq. (S3).

The parameters of Eq. (S2) are fitted on development missions. The mean is estimated by least squares, and the spread by least squares on the absolute residuals scaled by $\sqrt{\pi/2}$, which is less sensitive to the heavy tail of the errors than squared residuals. The latency $\bar{L}$ is the mean of the latency distribution in tier B and the measured mean plan-acceptance latency in tier C.

S2. Scenario generator, error models and exact-execution snapshots

S2.1. Scenarios

Missions are created by a seeded generator with the parameters in Table S1. The map is a square with a side of 10 km and the depot at its centre, and half of the tasks are clustered around three hotspots. Half of the tasks are known at the start, and the others arrive in batches. Three fleet scales with 5, 10 and 20 UAVs and 25, 50 and 140 tasks are combined with three event profiles. The arrivals profile contains six arrival batches, the failures profile three batches and two UAV failures, and the deadlines profile three batches and four tightened deadlines. A fourth scale with 40 UAVs and 280 tasks is used for scalability only.

Table S1: Scenario generator parameters (distances in m, times in s, energy in metres of flight).

Map side / horizon	10 000 / 1 800	Service time	$U[30, 90]$
UAV speed	$U[10, 16]$ m/s	Time-window slack	$U[180, 600]$
Energy capacity	$U[18, 28]$ km	Release delay	$U[0, 120]$
Energy reserve	10% of capacity	Rewards 1, 2, 3, 5	probabilities .4, .3, .2, .1
Hover cost	6 per second	Hotspots	3, std. 600, 50% of tasks
Tasks known at start	50%	Scales (UAVs/tasks)	5/25, 10/50, 20/140, 40/280

Table S2: Finish-time error models fitted on development missions: $m(h) = b_0 + b_1 h$, $s(h) = s_0 + s_1 h$ (seconds of the respective tier; n executed tasks).

Tier	Level	n	b_0	b_1	s_0	s_1
B	low	5399	8.91	−0.0044	0.00	0.0904
B	mid	4991	16.84	0.0354	1.58	0.1951
B	high	4253	20.79	0.2923	0.00	0.6389
C	low	90	−1.23	0.0281	1.30	0.0545
C	mid	89	2.36	0.0297	0.00	0.2303

S2.2. Static snapshots with exact execution

The stopping rules are first compared without execution uncertainty. Each mission is simulated once with a deterministic reference planner, and every replanning call after the start is stored with its previous plan. All methods are evaluated on these snapshots, and regret is measured against the best known objective of each snapshot, which includes a 3 s ALNS run and, for the small and medium scales, the OR-Tools routing solver [3] with guided local search [4]. Because all ALNS variants follow the same search trajectory, whole families of each rule are evaluated exactly offline, with 12 fixed budgets, 10 stagnation patience values and 9 compute prices, and each baseline is compared with AnytimeSwarm at the same mean computation. This benchmark uses its own pre-registered test seeds 2000–2029 (1562 snapshots of 270 missions), and it was reproduced on the final software platform before the execution-aware study (Section S3.1).

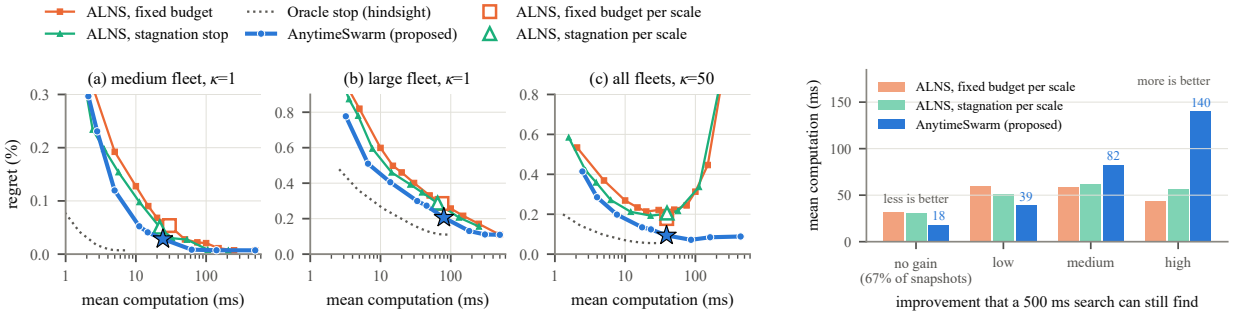
S3. Exact execution: stopping rules at equal computation

On the 1562 test snapshots, regret-2 repair finished within about one millisecond but left a regret of 0.98%. With a budget of 250 ms, ALNS with the full budget reached 0.06%, whereas AnytimeSwarm reached 0.08% with 16% of the budget, that is, with 84% less computation. Because the stopping rules use different amounts of computation, they are best compared at equal computation. Table S3 makes this comparison for each fleet scale, with the fixed budget and the stagnation rule evaluated at the mean computation that AnytimeSwarm used on the same scale. AnytimeSwarm reached a lower regret on every scale, and all paired intervals lie below zero. The reduction was 33% to 52% on the medium scale, 22% to 27% on the large scale and 12% to 20% with 40 UAVs, a scale that was never used for training. Read in the other direction, the two baselines needed between 1.5 and 2.8 times the computation of AnytimeSwarm to reach its regret. Over all scales, AnytimeSwarm reached 0.083% against 0.112% to 0.130% for four baselines that include a fixed budget and a stagnation patience tuned per fleet scale on development data.

Fig. S1 shows where this advantage comes from. On the medium and large scales, the quality–compute frontier of AnytimeSwarm lies below those of fixed budgets and stagnation stopping over the range of practical computation (Fig. S1a). Grouped by the improvement that a 500 ms search can still find (Fig. S1b), AnytimeSwarm spent 18.2 ms on snapshots that cannot be improved and 140.0 ms on those with the largest available improvement. Only 29% of its computation went to snapshots without any possible gain, compared with more than half for the baselines. The learned predictor is essential for this allocation. It reached an area

Table S3: Regret at equal computation per fleet scale (UAVs/tasks). Baselines run for the mean time of AnytimeSwarm on the same scale; †: 95% interval of the paired difference below zero. Right: computation a baseline needs to match AnytimeSwarm.

		Regret at equal computation (%)			Computation to match (ms)	
Scale	Time (ms)	Proposed	Fixed	Stagnation	Fixed	Stagnation
Small (5/25)	18	0.006	0.030†	0.029†	50 (2.7×)	51 (2.8×)
Medium (10/50)	25	0.030	0.062†	0.045†	49 (2.0×)	39 (1.5×)
Large (20/140)	81	0.213	0.290†	0.274†	172 (2.1×)	135 (1.7×)
40 UAVs (40/280)	151	0.205	0.256†	0.234†	299 (2.0×)	258 (1.7×)
Small to large	42	0.083	0.141†	0.127†	143 (3.4×)	99 (2.4×)



(a) (b)

Figure S1: Exact execution, test seeds 2000–2029: (a) quality–compute frontiers on identical search trajectories (open markers: baselines tuned per scale; star: selected AnytimeSwarm setting); (b) mean computation per snapshot, grouped by the improvement that a 500 ms search can still find.

under the ROC curve [5] of 0.926 for the event that the next 10 ms bring an improvement, compared with 0.757 for an extrapolation of the recent improvement rate.

S3.1. Reproduction on the evaluation platform

The exact-execution benchmark was produced on Windows. Before the execution-aware study, its development pipeline was re-run on the platform of all later experiments (Ubuntu 24.04 under WSL2, identical Python and package versions). The deterministic part was reproduced bit for bit, except for differences of one unit in the last place of some energy values, which stem from the floating-point libraries of the two operating systems. The search ran 21% more iterations per millisecond on the new platform. The improvement predictor, re-trained on the new timing traces, reached the same compute price by the pre-registered rule, and the conclusions of the equal-computation comparison agreed in sign and significance in 24 of 24 comparisons. The frozen predictor was used unchanged in all later experiments.

S4. Study 1: execution-aware stopping on the nominal planning model

Study 1 compares AnytimeSwarm-R with the stopping rules of the first group of Section 5.3 on the nominal planning model, on held-out seeds 3000–3029 in tier B (270 missions over three scales and three profiles, and 90 missions with 40 UAVs) and 4000–4009 in tier C (30 missions per method). Its primary matrix crosses two execution levels (low, mid), two latency levels (none, high), two latency factors ($\kappa_B = 10, 50$) and two triggers (event, periodic every 60 s). Its primary hypothesis is that AnytimeSwarm-R serves more reward than ALNS with 40 ms, AnytimeSwarm and the stagnation rule.

S4.1. Tier B primary matrix

The primary matrix comprises 34 560 mission runs, that is, 270 held-out missions in 16 configurations with eight methods. Table S4 lists utility, lost tasks and computation per method, and Fig. S2 relates utility to computation and shows the difference in every configuration.

Search pays off clearly under uncertain execution. Regret-2 repair reached a utility of 68.86%, and AnytimeSwarm-R reached 72.25%, which is +3.39 percentage points (pp) or 4.9% more. The full budget of 250 ms was the weakest search method, with 71.48%, and AnytimeSwarm-R served +0.77 [+0.31, +1.25] pp more reward than it. The loss of the long budget stems from late plans. A task counts as lost to delay when it is part of the computed plan but can no longer be served on time when the plan is released. With $\kappa_B = 50$, the full budget lost 5.36 tasks per mission in this way, against 1.28 for AnytimeSwarm-R, that is, 76% fewer. Without its delay cost, AnytimeSwarm lost 3.50 tasks per mission in this way, but its utility was not lower (72.13% against 72.04%). The delay cost thus prevents losses from late plans without a measurable gain in utility on this matrix.

Among the remaining search methods, the differences are small. All seven search methods lie within 0.77 pp of each other. AnytimeSwarm-R reached the highest mean utility with the least computation of the adaptive rules, 41.1 ms per call against 48.2 ms for AnytimeSwarm and 59.9 ms for the stagnation rule. The reduction of 15% relative to AnytimeSwarm is significant (−7.1 [−7.7, −6.5] ms per call), so the second hypothesis holds. The primary hypothesis does not hold at the pre-registered level. The pooled advantage of AnytimeSwarm-R was +0.32 [+0.01, +0.63] pp over ALNS with 40 ms, +0.21 [−0.08, +0.52] pp over AnytimeSwarm and +0.28 [−0.11, +0.67] pp over the stagnation rule, with Holm-adjusted p-values of 0.128, 0.308 and 0.308. Only the first interval excludes zero before correction. Per configuration and without correction for multiple testing, AnytimeSwarm-R was better than ALNS with 40 ms in 2 of the 16 configurations, better than the stagnation rule in 1 and never different from AnytimeSwarm, which is consistent with chance (Fig. S2b). The ablation that keeps only the latency term reached 72.02%, close to AnytimeSwarm. Together with the misspecification runs below, this suggests that a non-zero error spread mainly makes the rule stop earlier, rather than that the calibration of the spread matters.

The fixed budget of 40 ms is a strong reference, because it was matched to the mean computation of AnytimeSwarm on development data. Its weakness lies in the allocation across fleet scales. AnytimeSwarm-R used 24 ms per call on the small scale and 75 ms on the large scale. On the large scale, it served +0.59 [+0.34, +0.82] pp more reward than the budget of 40 ms and was on a par with the full budget (−0.06 [−0.38, +0.26] pp). On the small scale, it served +1.79 [+0.65, +3.00] pp more than the full budget and was on a par with the budget of 40 ms (+0.33 [−0.40, +1.09] pp). A single fixed budget is therefore either too long for small fleets or too short for large ones, whereas the adaptive rules find a suitable amount of computation on each scale without tuning. The advantage over ALNS with 40 ms was +0.43 [+0.11, +0.76] pp for $\kappa_B = 10$ and +0.20 [−0.19, +0.61] pp for $\kappa_B = 50$, and +0.36 [+0.01, +0.73] pp under low and +0.27 [−0.13, +0.66] pp under mid disturbances.

S4.2. Replanning triggers

The trigger mattered more than the stopping rule. Replanning every 60 s in addition to the events raised the number of planning calls from 6.8 to 24.2 per mission and the utility of every method, by +1.29 pp to +1.56 pp, with all intervals above zero (+1.56 [+1.26, +1.84] pp for AnytimeSwarm-R, +1.47 [+1.19, +1.75] pp for ALNS with 40 ms). The third hypothesis therefore holds. Periodic replanning probably helps because it corrects the accumulated deviations of the executed routes, which the event trigger only notices at the next event. The advantage of AnytimeSwarm-R over ALNS with 40 ms was +0.27 [−0.07, +0.62] pp with the event trigger and +0.36 [+0.03, +0.70] pp with the periodic trigger. The two values are close, and the protocol did not test their difference.

Table S4: Tier B primary matrix: 270 held-out missions × 16 configurations. Utility and lost tasks per mission, mean planning time per call, and pooled paired difference of AnytimeSwarm-R minus each method (percentage points, 95% bootstrap interval; Holm-adjusted p for the three primary comparisons).

Method	Utility (%)	Late	Skipped	Calls	ms/call	AnytimeSwarm-R minus method (pp)
Regret-2 repair	68.86	6.64	18.35	15.6	1.0	+3.39 [+2.76, +4.04]
ALNS, 250 ms	71.48	6.15	20.58	15.5	250.1	+0.77 [+0.31, +1.25]
ALNS, 40 ms	71.93	6.65	20.97	15.5	40.1	+0.32 [+0.01, +0.63], p_{Holm} 0.128
ALNS, stagnation	71.97	6.48	20.92	15.5	59.9	+0.28 [−0.11, +0.67], p_{Holm} 0.308
AnytimeSwarm w/o delay cost	72.13	6.35	20.49	15.5	54.1	+0.12 [−0.20, +0.45]
AnytimeSwarm	72.04	6.64	21.32	15.5	48.2	+0.21 [−0.08, +0.52], p_{Holm} 0.308
AnytimeSwarm-R, latency only	72.02	6.65	21.31	15.5	48.2	+0.23 [−0.06, +0.54]
AnytimeSwarm-R (proposed)	72.25	6.56	21.15	15.5	41.1	–

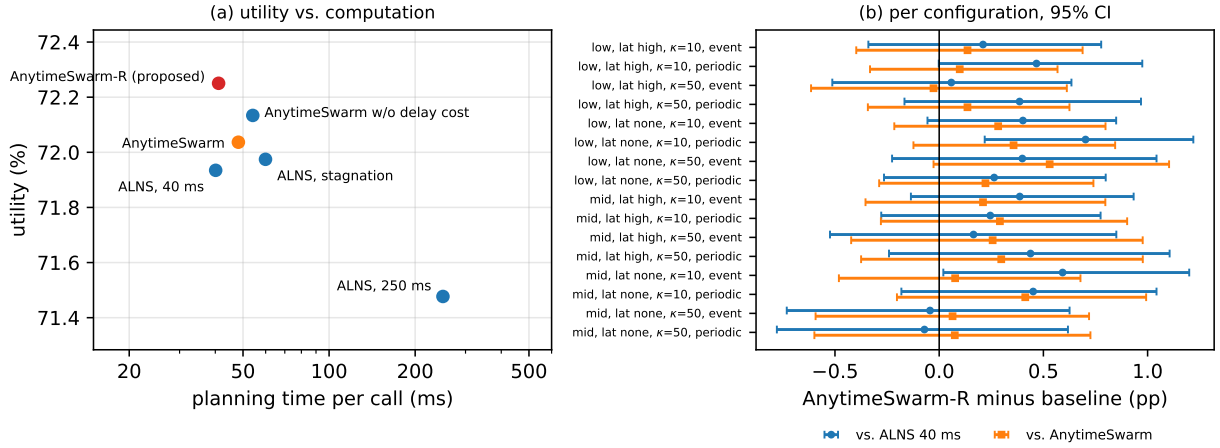


Figure S2: Tier B primary matrix, held-out seeds 3000–3029: (a) mean utility against mean planning time per call over all 16 configurations (regret-2 repair, at 68.86% and 1 ms, is not shown); (b) paired difference of AnytimeSwarm-R minus ALNS with 40 ms and minus AnytimeSwarm per configuration (execution level, latency level, κ_B, trigger) with 95% bootstrap intervals over 270 missions.

S4.3. Scalability to 40 UAVs

With 40 UAVs and 280 tasks, 90 held-out missions were run under mid disturbances in 4 configurations of latency level and latency factor. All five methods reached between 68.82% and 69.23%. AnytimeSwarm-R reached 69.16% with 127 ms per call. Its pooled difference was +0.02 [−0.26, +0.30] pp to ALNS with 180 ms, +0.34 [+0.05, +0.62] pp to AnytimeSwarm and −0.07 [−0.30, +0.16] pp to the stagnation rule. After Holm correction over the four comparisons of the pre-registered analysis, none of them is significant (p_{Holm} = 0.086 for AnytimeSwarm), so the fifth hypothesis is not supported either. AnytimeSwarm-R again needed less computation than AnytimeSwarm (−9.0 [−11.7, −6.2] ms per call). The stagnation rule stopped earliest, after 100 ms, and was significantly better than AnytimeSwarm-R in 1 of the 4 configurations, which had no latency and κ_B = 50. On this scale, the rules used between 100 and 250 ms per call, and utility hardly changed within this range of computation. The equal-computation advantage that AnytimeSwarm showed with 40 UAVs under exact execution (Table S3) therefore did not translate into a utility gain under uncertain execution.

S4.4. Sensitivity analyses

The sensitivity runs use the held-out missions of the primary matrix without latency and are descriptive. They vary the execution level beyond the primary matrix, the latency factor and the trigger in 8 configurations, and the error model in 4 further configurations.

Under high disturbances, which lie outside the range of the primary matrix but within that of the fitted error models, utility fell to about 48.93% for all methods. The combination of high disturbances and a slow planner ($\kappa_B = 50$) gave the largest advantage of the risk-aware cost in this study. AnytimeSwarm-R served +0.96 [+0.29, +1.66] pp more reward than AnytimeSwarm and used 49 ms instead of 58 ms per call. Its difference to ALNS with 40 ms was +0.14 [−0.53, +0.84] pp and to the stagnation rule +0.43 [−0.33, +1.21] pp, so the gain over AnytimeSwarm brought it to the level of the baselines rather than beyond them. With $\kappa_B = 1$, planning time hardly delays the plan, both delay costs are close to zero because they scale with the shift $\kappa\tau$, and the two rules behave alike. AnytimeSwarm-R then differed from AnytimeSwarm by +0.01 [−0.13, +0.15] pp and −0.5 ms per call.

Replanning when a UAV deviated by more than 30 s from its planned finish time raised the number of calls from 6.8 to 13.2 per mission and the utility by +0.86 [+0.58, +1.14] pp for AnytimeSwarm-R and +0.86 [+0.54, +1.18] pp for ALNS with 40 ms. Like the periodic trigger, it helped both stopping rules by a similar amount. Under the deviation trigger, the difference of AnytimeSwarm-R was +0.10 [−0.30, +0.52] pp to ALNS with 40 ms and +0.29 [−0.05, +0.65] pp to AnytimeSwarm.

The utility of AnytimeSwarm-R was insensitive to the error model. With the error model of the low, mid or high level under low and mid disturbances, its difference to AnytimeSwarm ranged between +0.07 [−0.31, +0.47] pp and +0.18 [−0.18, +0.55] pp, the correctly specified model was not better than a wrong one, and the mean utilities of all versions lay within 0.11 pp. Setting the error model to zero gave −0.13 [−0.32, +0.03] pp. The four misspecification configurations also belong to the primary matrix, so the correctly specified method was run twice on the same missions. The two runs differ only in the measured planning times, because the stopping rule reads a wall clock. They gave identical utility in 73% of the missions, and the mean utility of a method differed by up to 0.12 pp between the runs. Differences of this size between stopping rules should therefore not be read as systematic.

S4.5. Tier C: PX4 software-in-the-loop with Gazebo

All 120 held-out flights of tier C were completed, that is, 30 missions with four methods each. Two runs did not reach their altitude within 90 s and were repeated as specified. A further 4 runs had a real-time factor between 1.43 and 1.52, below the validity range of 1.6 to 2.2 stated in the protocol. The protocol did not specify how to treat such runs, so they were repeated once with the same seed, like a technical failure, and the original runs were kept. The repeated runs reached the same utility as the original ones, and the comparisons below are identical with either set (−0.51 [−3.24, +2.18] pp and +0.48 [−2.72, +3.46] pp with the original runs). The real-time factor of the analysed runs was 1.95 on average and at least 1.67. A released plan reached all vehicles after 0.152 s of simulation time on average, which is included in the expected latency of AnytimeSwarm-R.

Table S5 summarises the results. AnytimeSwarm-R reached a utility of 54.18%, against 54.69% for ALNS with 40 ms, 53.70% for AnytimeSwarm and 51.88% for the full budget. The paired differences were −0.51 [−3.24, +2.18] pp to ALNS with 40 ms and +0.48 [−2.72, +3.46] pp to AnytimeSwarm, with Holm-adjusted p-values of 1.000 and 1.000. The fourth hypothesis is therefore not supported. With 30 missions, the intervals are much wider than in tier B and cannot resolve differences of the size seen there. Two effects of tier B reappear in tier C. The full budget again had the lowest mean utility, although its interval includes zero (+2.30 [−1.32, +5.71] pp to AnytimeSwarm-R), and it was the only method that lost a noticeable number of tasks to late plans, 0.43 per mission against at most 0.03 for the others. AnytimeSwarm-R again computed least among the adaptive rules, with 34.7 ms per call against 47.4 ms for AnytimeSwarm, a reduction of 27% (−12.7 [−18.8, −7.2] ms).

Table S5: Tier C (PX4 SITL, Gazebo, ROS 2): 30 held-out missions per method, mid disturbances, $\kappa_B = 50$. Utility, tasks lost to late plans and skipped at departure per mission, planning time per call; tier B utility on the same rescaled missions for comparison.

Method	Utility C (%)	Lost to delay	Skipped	ms/call	Utility B (%)
ALNS, 250 ms	51.88	0.43	0.57	250.1	50.17
ALNS, 40 ms	54.69	0.03	0.50	40.1	51.72
AnytimeSwarm	53.70	0.03	0.43	47.4	51.90
AnytimeSwarm-R (proposed)	54.18	0.00	0.37	34.7	52.55

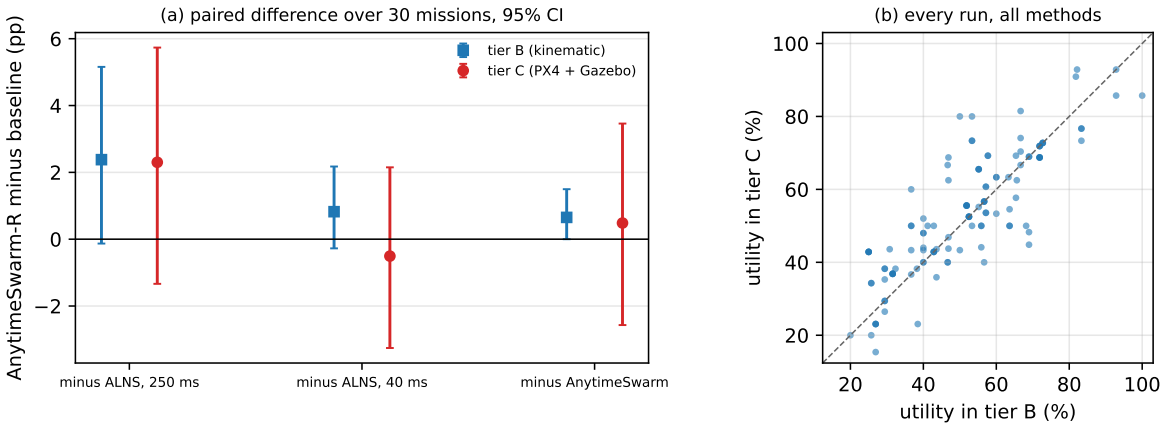


Figure S3: Tier B against tier C on the same 30 held-out missions (three UAVs, mid disturbances, $\kappa_B = 50$): (a) paired difference of AnytimeSwarm-R minus each baseline with 95% bootstrap intervals; (b) utility of every run in tier C against the same run in tier B (dashed: equality).

The same 30 missions were also run in tier B with the rescaled parameters of tier C. Tier B predicted the outcome of a flight well. The utility of a run in tier C correlated with that of the same run in tier B with $r = 0.84$ (120 runs), and the mean utility per mission with $r = 0.89$ (Fig. S3b). Tier C served slightly more reward on average, 53.61% against 51.58%, so the kinematic model of tier B was, if anything, slightly pessimistic for these missions. The small differences between the methods were not predicted as well. The method means of the two tiers had a Kendall rank correlation of 0.33, and the sign of the per-mission difference between AnytimeSwarm-R and ALNS with 40 ms agreed in 40% of the missions, counting ties as agreement only when both differences are zero. Part of the per-run correlation is by construction, because both tiers draw the wind, speed and service disturbances from the same random numbers, and much of it reflects the different difficulty of the missions. The correlation was first computed with an erroneous pairing of runs in the analysis script, which was corrected after the results had been seen; the hypothesis tests were not affected. In tier B, the advantage of AnytimeSwarm-R on these missions was +0.82 [−0.27, +2.16] pp over ALNS with 40 ms and +0.65 [+0.00, +1.50] pp over AnytimeSwarm (Fig. S3a). Both tiers thus agree that the methods differ by at most a few percentage points on this scale, and that the full budget is the weakest search method, while the order of the three short-computation methods is within the noise of both tiers.

S5. Study 2: regimes and software-in-the-loop details

Fig. S4 shows the paired differences of Table 3 of the main paper together with the computation of every method.

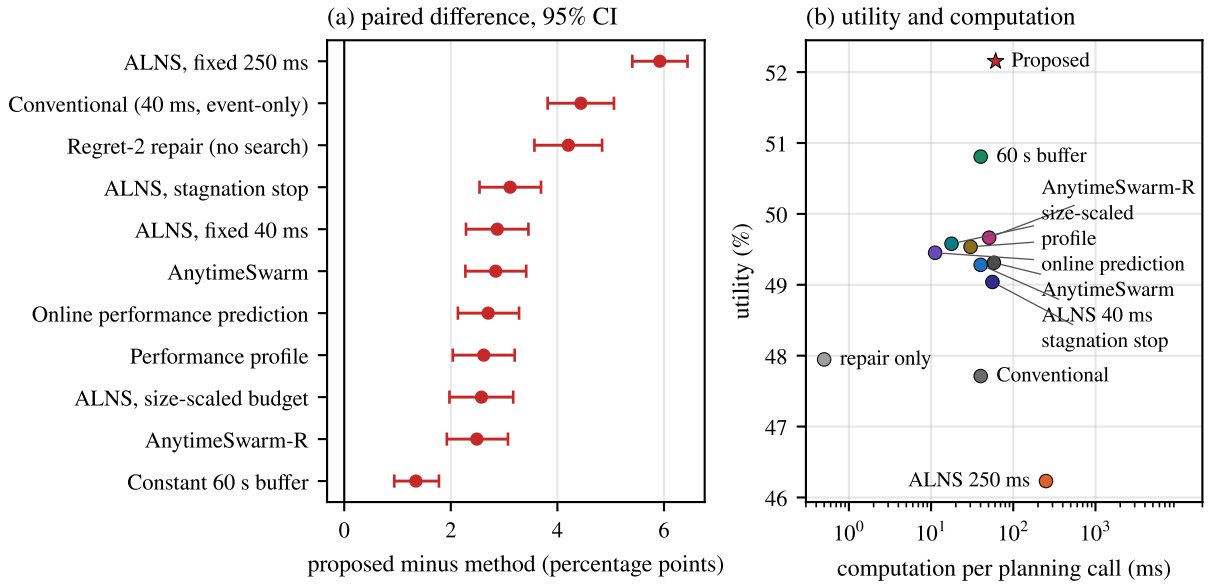


Figure S4: Study 2, tier B, held-out seeds 5000–5029: (a) utility of the proposed replanner minus each method, pooled over 12 configurations, with 95% paired bootstrap intervals over 270 missions; (b) mean utility against mean computation per call. Ablations are omitted.

S5.1. Regimes

Table S6 and Fig. S5 break the utility down by disturbance level, time-window factor and latency factor. Per configuration and without correction for multiple testing, the proposed replanner was better than ALNS with 40 ms in 8 of the 12 configurations and worse in 2, better than the constant buffer in 9 and worse in 3, and better than the conventional planner in 10 and worse in 0. The advantage over ALNS with 40 ms was +2.58 [+1.93, +3.22] pp, +2.71 [+2.10, +3.33] pp and +3.33 [+2.74, +3.91] pp for $\kappa_B = 10$, 50 and 200, and the advantage over online prediction stayed between +2.76 [+2.15, +3.36] pp and +2.57 [+1.97, +3.16] pp.

The advantage was largest with the original windows. Under high disturbances, the proposed replanner served +6.22 [+5.17, +7.29] pp more reward than ALNS with 40 ms and +3.03 [+2.01, +4.11] pp more than the constant buffer. The losses all occurred under high disturbances with halved windows. In this regime, the proposed replanner reached 33.38%, −2.01 [−2.80, −1.21] pp relative to the constant buffer and −1.57 [−2.52, −0.64] pp relative to ALNS with 40 ms. It served 24.3 tasks per mission, of which 6.2 late, against 25.3 and 8.4 for the constant buffer. The chance-constrained model was thus too cautious when the windows were short compared with the predicted finish-time error. The effective deadline of many tasks then probably falls before the earliest possible arrival, so the planner gives up tasks that a less cautious plan still serves in time. The value of z was selected per disturbance level on development missions, averaged over both window widths. Selecting it also per window width, or adapting it online, is a natural extension.

S5.2. Tier C: PX4 software-in-the-loop with Gazebo

All 120 held-out flights of study 2 were completed, that is, 30 missions with four methods each. One run failed to start and was repeated. As specified in the protocol, 46 runs with a real-time factor outside 1.6 to 2.2 were repeated once with the same seed, and the original runs were kept. The laptop was also used for other work during the campaign, and many repeats ran slower than the original runs. Of the 120 analysed runs, 77 lie in the validity range. The real-time factor was 1.64 on average, with quartiles of 1.41–1.95 and a minimum of 0.83. Two sensitivity analyses that were not pre-registered therefore repeat the comparisons

Table S6: Utility (%) per regime: disturbance level, time-window factor and latency factor κ_B (10, 50, 200), tier B held-out missions of study 2. Bold: best of the five methods.

	mid, windows ×1			mid, windows ×0.5			high, windows ×1			high, windows ×0.5		
Method	10	50	200	10	50	200	10	50	200	10	50	200
Proposed	**72.1**	**71.8**	**71.0**	**47.8**	**47.7**	**47.4**	**56.3**	**55.9**	**55.7**	33.5	33.3	33.3
Constant 60 s buffer	70.1	70.1	69.1	45.6	45.4	44.5	53.1	53.0	52.7	**35.7**	**35.5**	**34.9**
ALNS, fixed 40 ms	67.0	66.5	65.8	47.1	46.5	44.4	49.9	49.6	49.7	35.4	35.3	34.2
ALNS, fixed 250 ms	67.4	66.0	60.6	46.6	43.4	31.0	50.8	50.3	46.0	35.2	33.1	24.4
Conventional	65.6	65.5	63.9	45.4	44.9	42.7	48.7	48.6	47.5	34.1	33.6	32.2

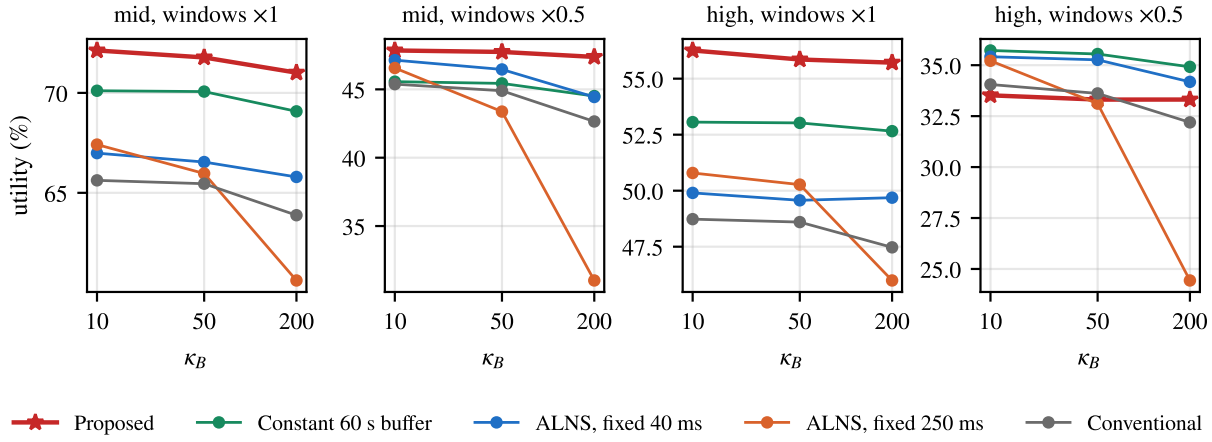


Figure S5: Study 2, tier B: utility per regime as a function of the latency factor κ_B, for mid and high disturbances with the original and with halved time windows (mean over 270 held-out missions per point). The proposed replanner is shown with stars.

with the original runs instead of the repeats, and with only those pairs of runs that both lie in the validity range.

Table 5 of the main paper summarises the results. The proposed replanner reached a utility of 59.89%, against 60.15% for ALNS with 40 ms, 57.26% for the constant buffer and 58.61% for the conventional planner. The paired differences were −0.25 [−4.02, +3.57] pp to ALNS with 40 ms and +1.28 [−2.20, +4.64] pp to the conventional planner, with Holm-adjusted p-values of 0.942 and 0.942. The fourth hypothesis is therefore not supported. The secondary comparison with the constant buffer gave +2.64 [−0.01, +5.58] pp. The sensitivity analyses lead to the same conclusion. With the original runs, the differences were −1.04 [−4.83, +2.77] pp and −0.02 [−3.66, +3.47] pp. With valid runs only, they were −2.02 [−6.25, +2.43] pp on 16 missions and −1.52 [−6.32, +3.37] pp on 14 missions. No interval in the three analyses excludes zero.

The mechanism seen in tier B reappears in the counts of late and skipped tasks. The proposed replanner served 0.90 tasks late per mission, against 1.20 for ALNS with 40 ms, which is 25% fewer. It skipped no task at departure, whereas ALNS with 40 ms skipped 0.23 and the conventional planner 0.57 tasks per mission. It computed 44.4 ms per call, against 40.2 ms for the fixed budget. The conventional planner called the search 6.3 times per mission instead of about 21.9, as in tier B.

The same 30 missions were also run in tier B with the rescaled parameters of tier C. As in study 1, tier B predicted the utility of a flight closely, with a correlation of $r = 0.86$ over 120 runs. On these missions, however, even tier B did not resolve the differences between the methods (Fig. S6). Its estimates were +2.82 [−1.20, +6.62] pp to ALNS with 40 ms, +1.90 [−0.59, +4.65] pp to the constant buffer and +1.80 [−2.31, +5.65] pp to the conventional planner. The tier C intervals overlap them. The per-mission difference to ALNS with 40 ms had a standard deviation of 10.9 pp in tier C. Confirming the tier B estimate of 2.8 pp on

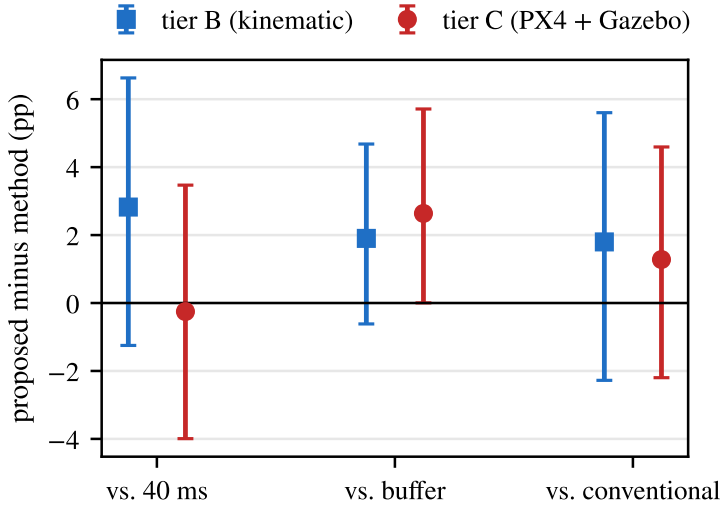


Figure S6: Study 2, tier B against tier C on the same 30 held-out missions (three UAVs, mid disturbances, $\kappa_B = 50$): paired difference of the proposed replanner minus each baseline with 95% bootstrap intervals.

these missions with a power of 80% would require about 118 missions per method. Tier C thus shows that the proposed replanner runs on an autopilot and a physics engine with the same computation and fewer late tasks, but it cannot confirm the size of the gain measured in tier B.

References

[1] P. McCullagh, J. A. Nelder, Generalized Linear Models, 2nd Edition, Chapman and Hall, London, 1989. doi:10.1007/978-1-4899-3242-6.

[2] M. W. P. Savelsbergh, The vehicle routing problem with time windows: Minimizing route duration, ORSA Journal on Computing 4 (2) (1992) 146–154. doi:10.1287/ijoc.4.2.146.

[3] L. Perron, V. Furnon, OR-Tools, https://developers.google.com/optimization, version 9.15 (2026).

[4] C. Voudouris, E. Tsang, Guided local search and its application to the traveling salesman problem, European Journal of Operational Research 113 (2) (1999) 469–499. doi:10.1016/S0377-2217(98)00099-X.

[5] T. Fawcett, An introduction to ROC analysis, Pattern Recognition Letters 27 (8) (2006) 861–874. doi:10.1016/j.patrec.2005.10.010.